MATLAB 완벽가이드



저자 : 임종수

이 문서는 한글97로 제작되었습니다. 하지만 도스용 한글3.0부터 윈도우용 한글97까지 완벽히 지원됩니다. 만약 그림부분이 나타나지 않는다면 RAM이 얼만큼인지 보시기 바랍니다. RAM용량이 적으면 나타나지 않을 수 있습니다. 또한 이 글을 보시게 될 분의 글꼴이 어떤지 모르기에 가장 기본이 되는 글꼴만 이용했음을 미리 알려드립니다.
기타 문의 사항이나 더욱더 알고 싶으신분은 다음의 연락처 혹은 홈페이지에 방문하시기 바랍니다.



하이텔 : ssljk07
E-mail : matlab@loveangel.net

제작·편집 : 임종관 「오정, 한글서당, MATLAB 전문학교, 이야기세상 운영자」

http://matlabschool.com [MATLAB 전문학교]

10월 6, 7일 이틀에 걸쳐...
임종수씨가 진행하는 세미나 개최!!

▶ 날짜와 안내

수강 인원 : 100명
세미나 날짜 : 2001년 10월 6일 (토) ~ 10월 7일 (일)
세미나 장소 : 서울 대학로 정보세미나홀 (자세한 안내도는 홈페이지에 기재)
세미나 시간 : 오전 10시 ~ 오후 6시 (12시 ~ 1시는 점심시간, 음료수 제공)
세미나 교재 : 임종수의 MATLAB 완벽가이드
(당일 시중가의 80%에 판매 - 시중가 18,000원 → 판매가 14,400원)
세미나 방식 : 5시간 강의후 2시간 질문/답변 시간

세미나 수강료 : 학생(하루 4만원, 이틀 7만원), 일반(하루 5만원, 이틀 9만원)
수강료 납부방식 : 선불이며, 무통장입금과 신용카드결제 두가지 방식으로 진행합니다.
수강신청 : http://matlabschool.com/seminar.html

▶ 강의 내용

첫째날(10월 6일 토요일)

1. matlab overview
2. matlab의 행렬 연산.
3. M file 작성 방법.
4. Matlab 그래픽스.
5. 파일입출력과 GUI programming 기본

둘째날(10월 7일 일요일)

1. Handle Graphics system.
2. Cell and struct data type.
3. 다양한 GUI Programming 방법.

※ 이번 강의는 MATLAB 5.x에 대한 것이며, 다음에는 6.x에 대한 강의를 진행할 예정입니다.
※ 이번 강의를 수강한 모든 분들께는 다음 강의를 수강료를 할인해 드립니다.



라. M-file 작성 시 유용한 함수들 :

이제, M-file작성 시 유용한 몇 가지 함수들을 알아보고, M-file programming.I을 끝내고자 한다.

1. input 함수 :

전형적인 사용법(syntax) : **input.m**

(a) "**R=input('prompt', 's')**"

→"**input.m**"**함수는 실행 시, 'prompt'** string**이 나오고,**

→**사용자가 문자열을 입력할 때까지 기다린다.**

→**사용자가 문자열을 입력하면, 그 입력된 문자열은**

→**출력 변수 R에 반환된다.**

→**단, 문자열이 수식을 표현하는 경우 계산되지 않는다.**

(b) "**R=input('prompt')**"

→"**input.m**"**함수는 실행 시, 'prompt'** string**이 나오고,**

→**사용자가 수식이나 변수이름을 입력할 때까지 기다린다.**

→**입력된 것이 수식인 경우에는** base workspace**에 있는 변수들에**

→**기반을 두고 계산하여 그 결과 값을 출력 변수 R에 반환한다.**

사용 예 :

```
□x=2; y=3; ↲
□R=input('아무 문자나 입력하시오! ','s') ↲
아무 문자나 입력하시오! test

R =

test

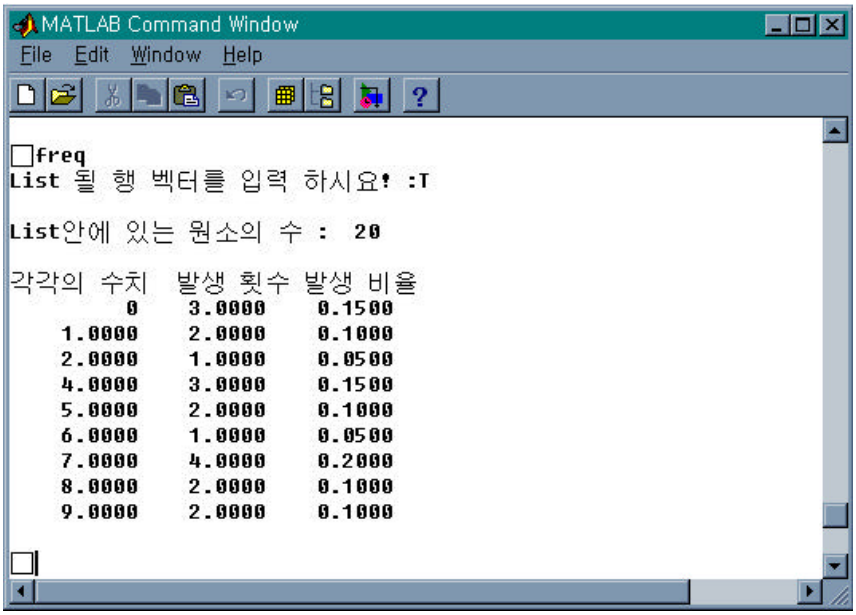
□R=input('가능 한한 변수 이름이나 수식을 입력하시오! ') ↲
가능 한한 변수이름이나 수식을 입력하시오! x+y

R =

5
```

특별히, "input.m"함수는 입력 변수가 한 개인 경우, 즉, "R=input('prompt')"인 경우, "prompt"다음에는 임의의 M-files를 포함한 수식이 나올 수 있고, 그 계산 결과를 반환한다. 만일, 여러 개의 결과 값을 반환하는 M-files가 "prompt"다음에 사용되는 경우, "input.m"함수는 해당하는 여러 개의 결과 값을 반환한다. 이 기능은 "R=input('prompt', 's')"의 형태(format)에서는 적용되지 않는다 데 주의하라!

[예제 18] 주어진 임의의 수치집합(numeric set)에서 어떠한 원소(또는, 사건(event))가 발생한 각각의 횟수와 발생 비율을 백분율로 [그림 2-39]와 같이 나타내는 함수를 작성해 보자.
주어진 임의의 집합 T=[2 0 7 7 9 4 5 8 0 1 5 7 0 4 1 4 7 6 9 8]이라고 하자.



[그림 2-39] [예제 18]의 실행 결과

[그림 2-39]의 실행 결과로부터 여러분은 이전에 배운 "disp.m"함수를 사용해야 된다는 것을 알 수 있을 것이다. 또한, 각각의 수치를 오름차순(ascending order)으로 정렬하는 함수가 필요하다. 물론, 여러분이 직접 데이터를 정렬하는 함수를 말들 수도 있겠지만. matlab에서는 "sort.m"라는 데이터 정렬 함수가 있다. 그 사용법은 다음과 같다.

전형적인 사용방법(syntax) : **sort.m**

(a)"**Y=sort(X)**"

→**벡터** X**를 오름차순**(ascending order)**으로 정렬하여 반환한다**.

(b)"**[Y, I]=sort(X)**"

→**벡터** X**를 오름차순**(ascending order)**으로 정렬하여 반환하고**,

→**각각의 원소가 벡터** X**에서 위치하는 곳을 출력 변수** I**에 반환한다**.

사용 예 :

```
X=[2 4 3 1]; ↲
[Y, I]=sort(X) ↲
Y =

     1     2     3     4  %벡터 X의 오름차순 정렬

I =

     4     1     3     2  %벡터 X에서의 원래 위치
```

그러나, 만일, X**가 행렬인 경우, 각각의 열**(column)**에 대해서 오름차순으로 정렬하고, 출력 변수가 두 개인 경우, 행렬** X**안에 있는, 각각의 열에 대한 원래의 위치를 반환한다. 이제,** [**예제** 18]**을 수행하는** M-file**을 작성할 모든 준비가 되었다**.

여러 가지 방법이 있겠지만, 필자는 우선 벡터 T**를 입력 변수로 하고, 오름차순으로 정렬된 벡터와 각각의 발생 횟수를 출력 변수로 하는 함수**(function mode M-file)**와** [**그림** 2-39] **와 같은 형태**(format)**로 출력하는** M-file(script mode M-file), **이렇게 두 개의** M-file**을 만들어서 해결 할 생각이다**.



주의 script mode M-file**과** function mode M-file**은 같은 이름의** M-file**안에 동시에 존재 할 수 없다.**

우선, function mode M-file**의 내용은** [**그림** 2-40a]**과 같다**.
파일 이름은 "csort.m"이다.

```
function d = csort (T)
%
%CSORT        벡터 T에 대한 발생 횟수와 중복이 없는 수치를 정렬하여 반환한다.
%
% 1993.8.4      By  Paul  E. Pfeiffer
% 1998.2.7      modified by Lim Jong_Su
%

T=T(:)';
P=ones(size(T));
D = [T;P];
[Y,I] = sort(T);
X = D(:,I);
m = length(T);
TI = X(1,:);
PI = X(2,:);
j = 1;
t(1) = TI(1);
p(1) = PI(1);
for i = 1:(m - 1)
 if abs(TI(i) - TI(i+1)) < 1e-6     %두 원소의 차가 1e-6보다 작으면
    j = j;                          %같다고 본다.
    p(j) =  p(j) + PI(i+1);
 else
   j = j+1;
   t(j) = TI(i+1);
   p(j) = PI(i+1);
 end
end
d = [t;p];
```

[그림 2-40a] "csort.m"함수의 내용

그렇게 어려운 내용은 아니지만, 위에서 13번째 줄, 즉, "**X = D(:,I);**"은 page20 맨 아래에 설명된 내용을 참고하면 쉽게 이해가 될 것이다. 꽤 유용한 기법이니 잘 알아두자.

[그림 2-40b]는 "**csort.m**"함수로부터 얻은 결과를 [그림 2-39]와 같이 screen에 표시 해주는 script mode M-file**이다**.

파일 이름은 "**freq.m**"이다. 여기서는 특별히 "**disp.m**"함수의 사용법에 주목하자.

```
%
%FREQ     함수 csort.m에의해 반환된 행 벡터에대해 목록을 list한다.
%!주의 : csort.m함수를 필요로 한다.
%
%1993.8.4     By  Paul E. Pfeiffer
%1998.2.7     modified by Lim Jong_Su
%

T = input('List 될 행 벡터를 입력 하시요! :');
n = length(T);
H = csort(T)';
A = [H H(:,2)/n];
disp(' ')        %한 줄 띄우기!
disp(['List안에 있는 원소의 수 :  ',num2str(n)] )
disp(' ')
disp('각각의 수치  발생 횟수 발생 비율')
disp(A)
```

[그림 2-40b] "freq.m" script mode M-file의 내용



필자는 여러분이 모든 예제들을 다 작성해보고, 또한, 이해를 하고 있다고 본다. 그러나, 혹시, 잘 이해가 가지 안는 부분이 있다면, 꼭, 질문을 하기 바란다. 대충 대충 넘어가는 분이 있다면, 후에 "professional matlab 21과정"은 상당히 어려운 내용이므로 결코, 이해하지 못 할 것이다.

2. "menu.m" 함수 :

전형적인 사용법(syntax) : **menu.m**

"**R=input('menu title', '선택1','선택2', ... ,'선택3')**"

→**주어진 'menu title'을 가진** menu window**를 생성한다**.

→**선택1, ... ,선택2 항목은 각각 "push" 버튼의** label**로 나타난다**.

→**임의의 "push"버튼을** click**하면 해당하는**

→**선택n의 번호를 반환한다**.

[**예제** 19] **배열 "t=-2*pi:1/100:2*pi"이고, "y=sin(t)"일 때, "menu.m"함수를 이용하여 선의 색을 "red, green, blue"중에서 하나를 선택할 수 있게 하고, "plot.m"함수로 도시**(plot) **할 수 있도록 작성하라**.

또한, 문자 'y'가 입력되면 실행이 중단 되도록 작성하라.

[**풀이**]

앞에서 배운 "input.m"함수와 "break.m"함수를 이용하여 코딩하였다.

[**그림** 2-41]**은 주어진 예제를 수행하는** M-file**의 내용이다**.

함수의 이름은 "ex2_16.m"이라고 하였다.

"**lower.m**"**함수는 입력된 문자를 소문자로 변환하여 반환한다**.

"**upper.m**"**함수는 입력된 문자를 대문자로 변환하여 반환한다**.

```
%
%이 예제는 menu.m함수 사용법을 설명하기 위한 것이다.
%
%1998.2.4     By Lim Jong_Su
%

line_color=['r','g','b'];
str='y';

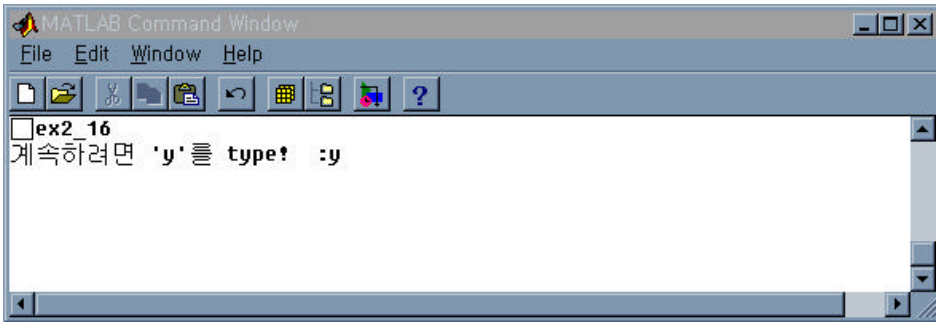
t=-2*pi:1/100:2*pi;
y=sin(t);

while(1)
   if str=='y'
      k=menu('선의 색을 결정','red','green','blue');
      plot(t,y,line_color(k))
      R=input('계속하려면 ''y''를 type!  :','s');
      str=lower(R);
   else
      break;
   end
end

close all
```

[그림 2-41]"ex2_16.m"파일의 내용

[그림 2-42a]는 "**ex2_16.m**"을 실행하였을 때의 command window 화면을 보여 주고 있다.



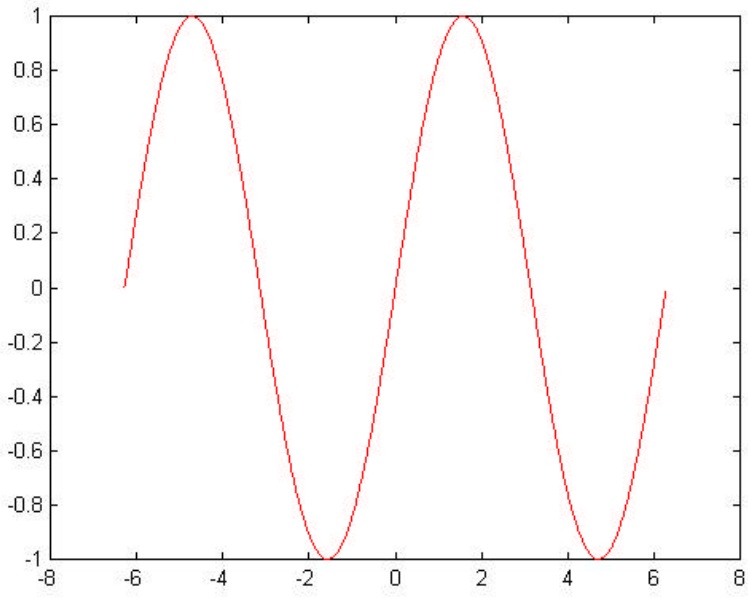
[그림 2-42a] "ex2_16.m"을 실행한 경우의 command window

[그림 2-42b]는 "ex2_16.m"을 실행하였을 때, 생성되는 "menu window"이다.
"push"버튼을 click하면, 선택한 색으로 선의 모양이 바뀐다.



[그림 2-42b]

[그림 2-42c]는 "ex2_16.m"을 실행하였을 때, 생성되는 plot인데, menu window에서 "red" 버튼이 click된 경우이다.



[그림 2-42c] "ex2_16.m"의 plot.

command window에 나오는 "prompt"에 'y'가 아닌 다른 문자를 type하면 실행을 중단한다.

3. dialog형(type) 함수들 :

여러 가지 dialog모양을 제공하는 함수들이 있는데, 그 사용법은 거의 비슷하다.
그래서, 여기서는 대표적으로, "questdlg.m"함수와 "msgbox.m"함수만 다루어 보겠다.

그 밖의 dialog형 함수들로는 다음과 같은 함수들이 있다.
helpdlg, inputdlg, pagedlg, printdlg, errordlg, warndlg


주의

dialog형 함수가 직접적으로 dialog형 함수를 제어하게 만들면 시스템에 따라서 프로그램을 불안하게 만든다. 즉, 실행 중 error가 발생 할 수 있다. 그러므로, dialog형 함수들 간에는 직접적인 데이터 교환은 하지 말도록 하자!

전형적인 사용방법(syntax): **questdlg.m**

```
R = questdlg('qstring','title','str1','str2','default')
```

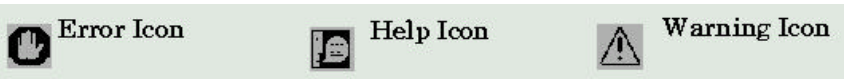
→질문형 dialog box를 만들고,
→두 개의 'str1','str2' labels를 가진 "push"버튼을 만든다.
→"default"버튼은 default상태를 지정하게 해 준다.
→default상태는'str1','str2' 둘 중에서 정한다.
→'str'은 하나 더 사용 할 수 있다.
→선택된 'str'이 출력 변수 R로 반환된다.

전형적인 사용방법(syntax): **msgbox.m**

```
msgbox(message,title,'icon')
```

→title을 box이름으로 하고, message를 표시하는 box를 만든다.
→message box안에 표현할 icon모양을 결정한다.

"msgbox.m"함수의 icon 종류 :
icon의 종류로는 보통 3가지를 사용하는데, 'error', 'help', 'warn'을 주로 이용한다. 아래 그림을 참고하기 바란다.



[그림 2-43] "msgbox.m"함수의 icon 종류

[예제 20] [예제 16]에서 사용한 "input.m"함수의 기능을 "questdlg.m"함수로 대체한 프로그램을 작성하시오.

[풀이]
[그림 2-44]은 "ex2_17.m"파일의 내용이다. 쉽게 이해 할 수 있다고 본다.

```
%
%이 예제는 menu.m, questdlg.m함수의 사용법을 설명하기 위한 것이다.
%
%1998.2.4     By Lim Jong_Su
%

line_color=['r','g','b'];
str='Yes';

t=-2*pi:1/100:2*pi;
y=sin(t);

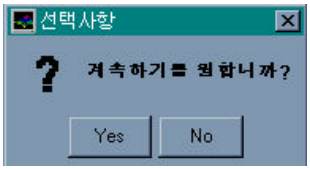
while(1)
   if strcmp(str, 'Yes')
      k=menu('선의 색을 결정','red','green','blue');
      plot(t,y,line_color(k))
      str=questdlg('계속하기를 원합니까?','선택사항','Yes','No','No');

   else
      break;
   end
end

close all
```

[그림 2-44]"ex2_17.m"파일의 내용

[그림 2-45]는 "ex2_17.m"파일을 실행하였을 때 생성되는 "questdlg.m"함수의 window이다. 여러분이 "Yes"를 누르면 계속해서 선의 모양을 바꾸어 줄 수 있다.



[그림 2-45]

[예제 21] page51 [예제 2]의 "pm_c.m"함수에 있는 "error.m"함수를 "msgbox.m"함수로 대체하여 다시 작성해 보시오.

[풀이]
독자 중에 한 분이 "pm_c.m"함수에 문제가 있다고, 지적을 해주었다. 그래서, "pm_c.m" 함수를 다시 작성하는 도중에, "msgbox.m"함수를 한 번 이용해 보았다.

[그림 2-46]은 다시 작성한 "pm_cb.m"함수의 내용이다.

```
function PM_cb(x)

%짝, 홀수를 판별하는 M-file.
%
%1998.2.4      By Lim Jong_Su

if x ~=abs(x)
   msgbox('입력 값은 양의 실수이어야 한다.','에러','error');
   return;
%  error('입력 값은 양의 실수이어야 한다.');

elseif x ~= fix(x)
   msgbox('정수가 아니다.','에러','error');
   return;
end

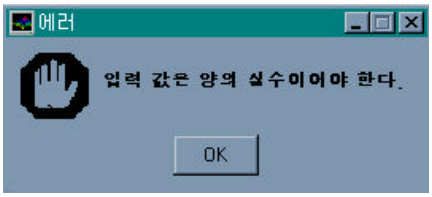
y=rem(x,2);

switch y
case 0
   disp('입력 값은 짝수!');
otherwise
   disp('입력 값은 홀수!');
end
```

[그림 2-46] "pm_cb.m"함수의 내용

[그림 2-47]는 "msgbox.m"함수가 생성하는 메시지 box인데, 다음과 같은 명령을 실행했을 때 발생하는 메시지 box이다.

□pm_cb(-2) ↲



[그림 2-47]

4. 그 밖에 많이 사용되는 함수들 :

대표적인 두 개의 함수를 설명하겠다.
이들 함수들은 앞으로 자주 이용하게 될 것이다.

전형적인 사용법(syntax) : **fliplr.m**
"**B=fliplr(A)**"
→배열 A에서 각 행의 원소 나열 순서가 좌우 뒤바뀐다.

사용 예 : X=[2 5 1 8 3]; ↲
Y=fliplr(X) ↲
Y =

3 8 1 5 2

유사한 함수로 상·하 순서가 뒤바뀌는 "flipud.m"함수도 있다.

마. P-code Files : matlab5.0이후 버전에만 있음

그 동안, M-file은 모든 소스를 다른 사람들이 쉽게 볼 수가 있어서, 프로그램을 보호 할 수가 없었다. 개인마다 다르겠지만, 힘들게 만든 프로그램을 다른 사람에게 그냥, 무상으로 제공하는 것은 어쩌면, 프로그램 개발의욕을 떨어뜨릴 수도 있다. 그래서, 나온 것이 "**pcode.m**"함수이다. 이 함수는 모든 M-files를 저장하기 전에 **parsing**하여 저장한다.


용어해설

parsing(구문 해석)

구문 구조를 해석하는 것. 컴퓨터 분야에 있어서는 컴파일러 프로세서 안의 구문해석 루틴동작을 가리켜서 파싱(parsing)이라고 한다.

matlab은 모든 M-files를 저장하기 전에 parsing하여 **pseudocode**로 변환하여 그 결과를 메모리에 저장한다.


용어해설

pseudocode(의사 코드)

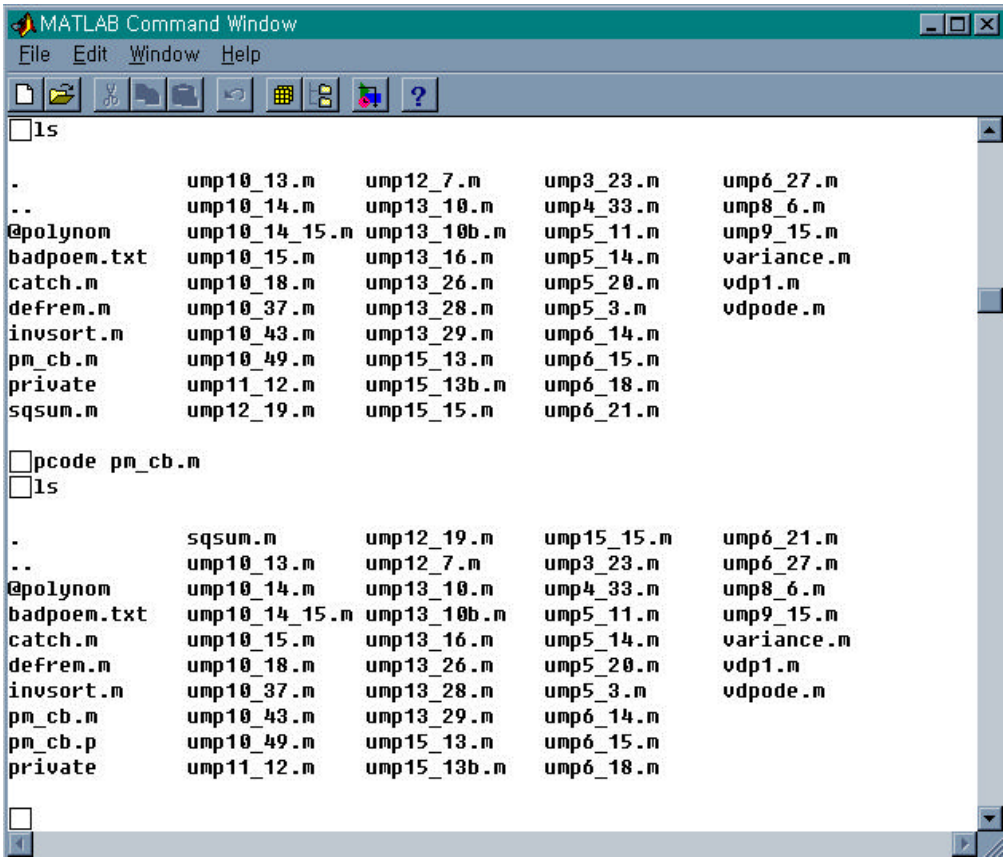
실행에 앞서서 번역을 필요로 하는 코드. 프로그램밍을 쉽게 하기 위해서 사용하는 대용 코드로서 실행하기 전에 기계코드로 번역된다.

"**pcode.m**"함수는 주어진 M-files를 "**pseudocode**"로 변환하여 확장자가 "**.p**"인 P-files로 저장한다. 물론, P-files도 M-files처럼 matlab에서 잘 수행된다. 또한, p-code화 한, M-files는 parsing이 이미 된 것이므로 재차 parsing을 하지 않는다. 그러므로, 대응하는 M-files보다 실행 속도가 더 빠르다.
그러나, matlab은 상당히 빨리 parsing을 수행하므로 P-files와 M-files의 실행 속도는 거의 같다고 봐도 된다. 만일, 여러분이 큰 GUI프로그램을 만든 경우에는 P-files가 빠르다는 것을 알 수 있을 것이다.

전형적인 사용방법(syntax) : **pcode.m**

(a) "**pcode fun1 fun2 ...**"
→주어진 함수1, 2를 p-files로 저장한다.
(b) "**pcode *.m**"
→현재 디렉토리에 있는 모든, M-files를 p-files로 바꾼다.

[그림 2-48]은 [예제 21]에서 만든 "pm_cb.m"함수에 대한 p-file을 만드는 과정을 보여 주고 있다.



[그림 2-48] "pm_cb.m"파일을 "pm_cb.p"파일로의 전환

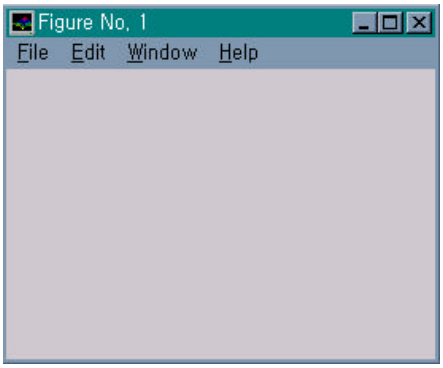
[그림 2-48]에서 알 수 있듯이 "pcode pm_cb.m"명령을 command window에서 실행하면 "pm_cb.p"파일이 새로 생긴다. 물론, "pm_cb.m"파일은 그대로 남아 있다. 그러므로, 남에게 소스나 알고리즘을 보여 주고 싶지 않으면, P-files를 주면 된다.

3. Matlab Graphics.

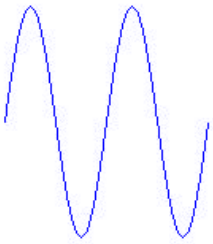
Mathworks회사에서는 matlab에서 이용되는 그래프들의 체계를 위해 "**Handle Graphics system**"이라는 새로운 용어를 만들어 냈다. 또한, 컴퓨터 screen을 포함한 개개의 모든 그림창(figure window)들과 좌표축들 그리고 patch(그림 성분)들은 "**graphics object**"라고 부른다. 즉, 앞으로 배울 line, text, image, user interface controls, 그리고, user interface pulldown menus 등등은 모두 "**graphics object**"이다.

"**graphics objects**"는 모두 9가지로 분류되는 데, Handle Graphics system에서 복잡한 그림을 만들기 위해 사용되는 성분들이라고 생각하면 좋을 것이다.

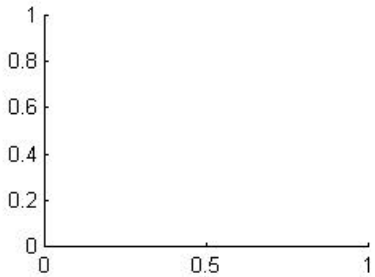
[그림 3-1]은 이들 "**graphics objects**"의 종류와 해당하는 예를 보여 주고 있다.



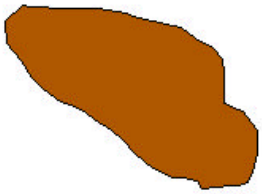
(a) figure object



(c) Line object



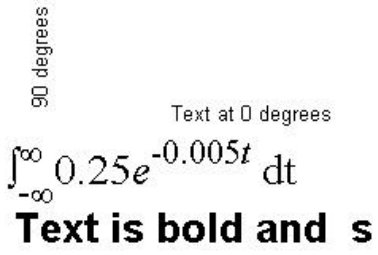
(b) Axes object



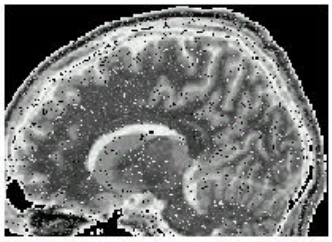
(d) Patch object



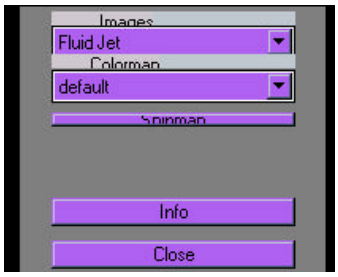
(e) Surface object



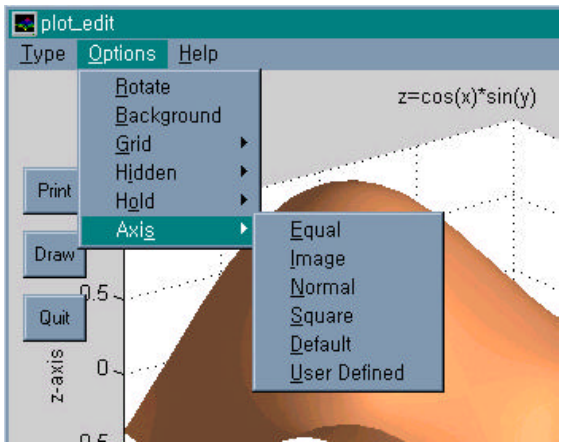
(g) Text object



(f) Image object



(h) Uicontrol object

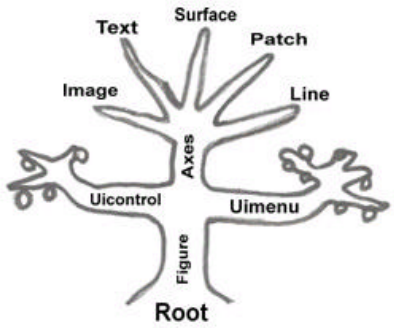


(i) Uimenu object

[그림 3-1] matlab object의 종류와 해당 예제 그림들.

이들 개개의 object(앞으로는 "객체"라고 이름 할 것이다.)를 정교하게 제어 할 수 있는 경우와 없는 경우에 의해서, 다시금 matlab 그래픽 함수들을 3가지로 분류 할 수 있다. 우선, 3가지 분류 사항을 배우기 전에 객체들의 계층 조직(hierarchy)을 이해 할 필요가 있다. 앞으로 여러분이 GUI 프로그램을 작성 할 때, 이들 객체의 의미와 또한, 객체간의 계층 조직을 이해해야 작성 할 수 있다.

사실, 개개의 객체에 대한 정확한 설명을 하려면, 상당량의 지면을 요구하므로, 강의를 진행해 나가면서 중간 중간에서 여러분이 이해 할 수 있도록 설명하는 식으로 해 나가겠다.
[그림 3-2]는 [그림 3-1]에서 보여준 객체간의 계층도 이다.



[그림 3-2] matlab 계층도

앞으로, [그림 3-1]과 [그림 3-2]는 자주 설명하는데, 인용될 것이다. 우선, [그림 3-2]에서 알 수 있듯이, **Root**가 가장 높은 level을 갖는다. 가장 높은 level을 갖는 다는 뜻은 어떠한 matlab graphics를 만들건 반드시 해당 그래프에 존재해야 한다는 뜻이다. 이것은 마치, [그림 3-2]에서 보여준 계층도의 나무에서 **Root**를 없앤다면 나머지 객체들 즉, Figure, Axes, Uimenu, 등도 모두 없어진다는 뜻이 된다. 그렇다면, 이처럼 모든 graphics에 반드시 있어야 하는 객체인 **Root**는 어떤 것을 의미할 까? 생각해 보기 바란다.

Root라는 객체가 의미하는 것은 바로, 여러분의 컴퓨터 **screen**이다. 만일, 컴퓨터 screen이 없다면, 어떠한, 그래프로 그릴 수 없기 때문이다.
그러므로, **상위 level을 지우면 하위 level의 객체는 자동으로 소멸된다.** 그러나, 하위 level을 지운 경우, 상위 level의 객체는 영향을 안 받는다. 즉, Axes를 없앤다고 해서 Figure 도 같이 사라지는 것은 아니다.

정리하면 다음과 같다.
1) matlab 그래프들은 객체들의 조합으로 만들어진다.
2) 객체들은 계층도에 의해 분류된다.
3) 상위 객체는 하위 객체들을 제어 할 수 있다. 그러나, 하위 객체는 상위 객체에 영향을 주지 못한다.
4) 객체들은 같은 level의 객체들(siblings)을 소유 할 수 있다.

자! 이제, 여러분은 이들 객체를 제어하는 방법을 알 고 싶을 것이다.


주의 조금은 난해한 부분이고, 또한, 내용이 상당히 많기 때문에 여기서 전부 이해하려고 하지 말고, 앞으로 제시할 예제를 통해서 차근차근 배워 나가도록 하자!

I. Handle Graphics.

각각의 matlab 그래프 함수는 객체들을 제어 할 수 있도록, 새로 만들어지는 객체마다 고유한 숫자를 반환한다. 이들 고유한 숫자들을 **handle**이라고 부른다.
handle은 보통 figure object인 경우에는 양의 정수를 반환하고, 다른 하위 level에서는 부동소숫점(floating point number)을 반환한다.

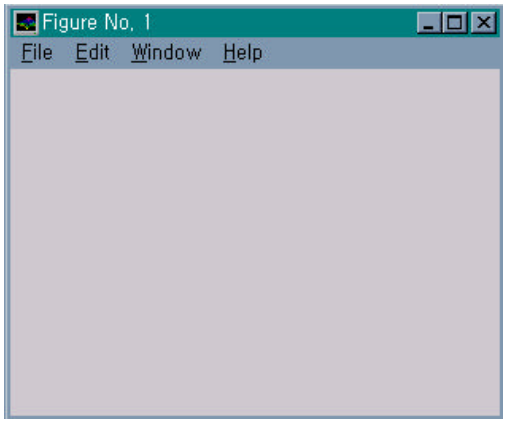
예를 들어보자.

```
□h_fig=figure ↲

h_fig =

     1
```

h_fig는 "1"이라는 양의 정수를 반환한다. 또한, 현재 열려 있는 figure가 없다면, 새로 figure를 만들어 낸다. 어쨌든, 바로 "1"이 생성된 figure의 handle value이다.
[그림3-3]은 현재 열려 있는 figure가 없는 경우, 새로 만들어 진 figure이다.



[그림 3-3] 새로 생성된 figure

[그림 3-3]에서 "**Figure No. 1**"이라는 글자가 보일 것이다. 여기서, "1"이 바로 h_fig값 "1"과 같은 값이다. 그러므로, "□h_fig2=figure(2) ↲"를 하면, "**Figure No. 2**"이라는 글자가 있는 새로운 figure가 생긴다.

일단, handle value가 할당된 객체는 자신의 handle value를 바꿀 수 없으며, 해당 객체가 사라질 때까지 handle value는 보존된다. 또한, 같은 종류의 객체라도 handle value는 서로 다르다. 그래야, 각각의 객체를 독립적으로 제어 할 수 있기 때문이다. 그러나, Root object의 handle value는 항상 "0"이다. 왜냐하면, matlab graphics를 만들 screen은 하나 뿐이기 때문이다.

만일, matlab에서 처음으로 "handle"과 "object"라는 용어(term)를 들은 분은 잘 이해가 가지 않을 지도 모른다. 그러므로, "handle"과 "object"에 대한 간단한 다른 예제를 다루어 보겠다.

먼저, handle을 이용하여 각각의 객체 성질(propertyname)을 조정 해주는 "set.m"함수와 해당 객체의 성질에 할당된 값(propertyvalue)을 얻게 해주는 "get.m"함수의 이용법을 간단히 알아보자.

전형적인 사용법 : set.m
"set(handle, propertyname, propertyvalue)"
→주어진 handle에 해당하는 객체의 propertyname을
→새로운 propertyvalue로 고쳐준다.

전형적인 사용법 : get.m
"propertyvalue=get(handle, propertyname)"
→주어진 handle에 해당하는 객체의 propertyname에
→대한 propertyvalue를 얻는다.

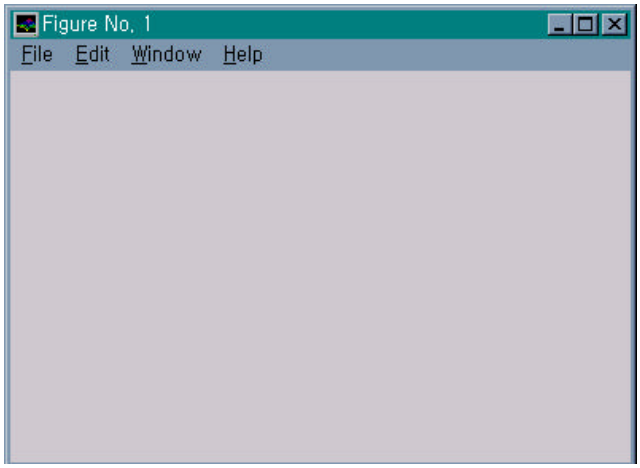
```
□close all                    %열려 있는 figure(active figure)를 모두 지운다.
□h_fig1=figure(1) ↲

h_fig1 =                      %"Figure No. 1"이라는 figure가 생길 것이다.

     1
```

[그림 3-4]에서 보여준 "Figure No. 1"을 "numbertitle"이라고 부른다. 이때, "numbertitle"을 주어진 figure의 "propertyname"이라고 하며, "Figure No. 1"을 "propertyvalue"라고 한다.

정리하면, 주어진 객체 figure의 성질(propertyname)들 중에서 "numbertitle"에 할당된 "propertyvalue"는 "Figure No. 1"이라는 뜻이다.



[그림 3-4] "numbertitle"이 "Figure No. 1"

이제, "**Figure No. 1**"대신에 "**By Lim Jong_Su**"라고 하고 싶다. 그러기 위해서는 우선, propertyname인 numbertitle을 없애고, 새로운 이름(**name**)인 "**By Lim Jong_Su**"을 할당해 주어야 한다. 여기서, 주의 할 것은 "numbertitle"과 "name"은 서로 다른 propertyname임이라는 것이다.

이 작업은 2단계로 이루어진다.

가. propertyname인 numbertitle을 없애기 위해서 propertyvalue에 "off"를 할당한다.
나. 새로운 이름을 할당하기 위해서 propertyname인 "name"에 대한 propertyvalue 에 "By Lim Jong_Su"를 할당한다.


주의

propertyname인 "**numbertitle**"에 대한 propertyvalue에 "**off**"대신 비슷한 단어인 "none, no, 등등"을 사용하면 다음과 같은 error가 나타난다.

```
??? Error using ==> set
Bad value for figure property: 'NumberTitle'.
```

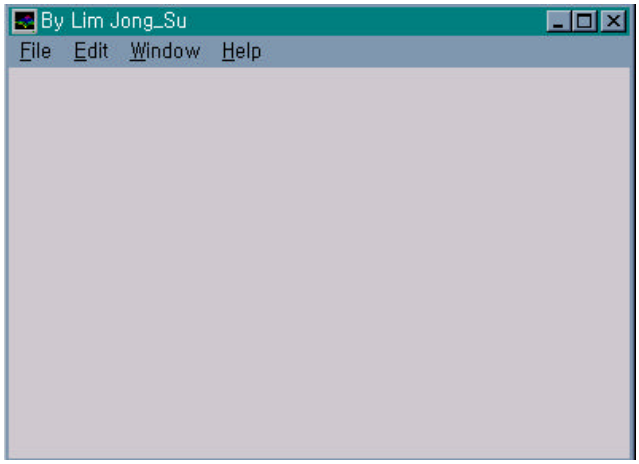
즉, "**numbertitle**"에 할당 할 수 있는 propertyvalue는 이미 지정되어 있는 "on 또는, off"뿐이다.

propertyname과 propertyvalue을 조정하기 위해서는 앞에서 설명한 "**set.m**"함수를 이용해야 한다.

□set(h_fig1,'numbertitle','off','name','By Lim Jong_Su') ↲

h_fig1	주어진 figure의 handle
numbertitle, name	주어진 figure의 propertyname
off, By Lim Jong_Su	주어진 figure의 propertyvalue

이제, 주어진 figure의 "**Figure No. 1**"은 "**By Lim Jong_Su**"로 바뀔 것이다.
[그림 3-5]에서 결과를 볼 수 있다.



[그림 3-5] "name"이 "By Lim Jong_Su"

만일, handle이 h_fig1인 figure의 크기를 알고 싶다면, 다음과 같이 하면 된다.

```
□h_size=get(h_fig1,'position') ↲

h_size =

    189    304    390    246
```

모든 객체의 propertyname과 propertyvalue는 여기서 알아본 방식으로 "set.m"함수와 "get.m"함수를 이용하여 조정한다.

객체 figure는 여기서 언급한 propertyname외에도 수많은 propertyname이 있고, 각각의 propertyname마다 할당 할 수 있는 propertyvalue가 지정되어 있다. 게다가, [그림3-1]에서 언급한 객체마다 서로 다르거나, 또는 공통인 객체가 많이 존재한다.
만일, 여러분이 훌륭한 matlab GUI프로그램을 작성하고 싶다면, 각각의 객체마다 존재하는 propertyname의 정의와 할당 할 수 있는 propertyvalue를 모두 기억해야하고 이해해야한다.
그렇다면, 객체 figure의 propertyname과 할당할 수 있는 propertyvalue의 종류는 무엇이 있을까?

□set(h_fig1) ↲
BackingStore: [{on} | off]
CloseRequestFcn
Color
Colormap
CurrentAxes
CurrentObject
CurrentPoint
Dithermap
DithermapMode: [auto | {manual}]
IntegerHandle: [{on} | off]
InvertHardcopy: [{on} | off]
KeyPressFcn
MenuBar: [none | {figure}]
MinColormap
Name
NextPlot: [{add} | replace | replacechildren]
NumberTitle: [{on} | off]
PaperUnits: [{inches} | centimeters | normalized | points]
PaperOrientation: [{portrait} | landscape]
PaperPosition
PaperPositionMode: [auto | {manual}]
PaperType: [{usletter} | uslegal | a3 | a4letter | a5 | b4 | tabloid]
Pointer: [crosshair | fullcrosshair | {arrow} | ibeam | watch | topl | topr | botl |
botr | left | top | right | bottom | circle | cross | fleur | custom]
PointerShapeCData
PointerShapeHotSpot
Position
Renderer: [{painters} | zbuffer]
RendererMode: [{auto} | manual]
Resize: [{on} | off]
ResizeFcn
ShareColors: [{on} | off]
Units: [inches | centimeters | normalized | points | {pixels}]
WindowButtonDownFcn
WindowButtonMotionFcn
WindowButtonUpFcn
WindowStyle: [{normal} | modal]
ButtonDownFcn
Children
Clipping: [{on} | off]
CreateFcn

DeleteFcn
BusyAction: [{queue} | cancel]
HandleVisibility: [{on} | callback | off]
Interruptible: [{on} | off]
Parent
Selected: [on | off]
SelectionHighlight: [{on} | off]
Tag
UserData
Visible: [{on} | off]

아마도, 조금 당황하였을 것이다. 단지 figure객체의 propertyname과 할당할 수 있는 propertyvalue의 조합이 이렇게 많은 데, [그림 3-1]에서 보여준 나머지 8개의 객체에 대한 propertyname과 propertyvalue를 합친다면, 9개 객체에 대한 propertyname과 propertyvalue를 전부 이해하고 기억한다는 것이 장난이 아니라는 것을 알 수 있을 것이다. 그러므로, 훌륭한 matlab GUI 프로그램을 만드는 것이 C++보다 훨씬 쉬운 것이 아니라는 것을 알아두자.

그러나, 필자의 생각으로는 본 강좌와 앞으로 쓸 "**professional matlab21과정**"만 착실히 배우면 괜찮은 GUI프로그램을 만들 수 있다고 본다. 왜냐하면, 필자는 현재 나와 있는 모든 matlab GUI프로그램을 만들 수 있다고 자신하기 때문이다.

II. Low_level functions.

원칙적으로 좀더 다루기가 쉬운 high level function(예: plot.m, mesh.m etc)을 먼저 강의하는 것이 보편적인 방법이지만, 필자의 생각으로는 Matlab graphics system의 기본인 handles를 정교하게 다룰 수 있는 low_level functions를 배우는 것이 Matlab의 graphic 체계를 전반적으로 이해하는 데 중요하다고 생각하여 low_level functions를 우선 다루기로 하였다.

low_level functions는 객체(Object) 지향적인 Matlab의 graphic 체계를 의미하는데, 이들 low_level graphic functions를 이용하여 모든 high_level graphic functions를 만들 수 있으며, 또한, 여러분이 원하는 다른 graphic functions를 작성 할 수도 있다.

Handle Graphic functions라고도 불리는 이들 low_level functions의 종류로는 [그림 3-1]에서 언급한 객체마다 해당하는 함수들 즉, 9가지가 있다. 본 단원에서는 이들 9가지 함수들을 전반적으로 자세하게 다루지는 않을 것이다. 왜냐하면, 이전에 언급하였듯이 각각의 함수에 해당하는 propertyname과 propertyvalue가 상당히 많고, 그렇게 자세하게 알아 둘 필요는 없다고 보기 때문이다. 그래서, 이들 함수에 대한 중요한 이용법만 설명하겠다. 또한, Matlab의 help 디렉토리에 HTML양식으로 또는, PDF양식으로 모두 설명되어 있으니 부족한 부분은 참고하기 바란다.


메모

앞으로의 내용 전개는 다소 복잡하고, 중요하게 보이지 안을지도 모른다. 그러나, "부록"에서 다룰 "GUI 프로그램 작성법"에서 필수적인 부분이다. 또한, "set.m"함수와 "get.m"함수의 이용법은 매우 중요하므로, 이번에 꼭 배워두기 바란다.

가. Root Object

Root Object에 해당하는 직접적인 함수는 없다. 또한, 다른 low_level functions보다 상당히 적은 propertyname/propertyvalue pairs를 갖는데, Matlab command window에서도 직접 얻을 수 있다.


메모

이제부터 설명하는 부분은 Root Object뿐만 아니라, 모든 object에 대한 내용이다 또한, 여러분이 Matlab manual을 보거나, GUI 프로그램을 작성 할 때, 자주 참고하여야 할 propertyname/propertyvalue에 대한 자료를 보는 방법이므로 자세히 알아 둘 필요가 있다.

□set(0) ↲

위와 같은 명령을 실행하면 Root Object에 대한 자료를 볼 수 있다. 이전에, figure object의 handles value는 항상 양의 정수라고 하였다. 그런데, Root Object의 handle value는 항상 "0"이라는 것에 주의하자! 또한, command window에서 display되는 Root Object에 대한 자료를 보는 데 있어서, 몇 가지 주의 사항이 있다.
나열되는 자료 중에 다음과 같은 항들이 있을 것이다.

```
*Diary: [ on | off ]                    ..... (줄 1)
*ShowHiddenHandles: [ on | {off} ]      ..... (줄 2)
```

(줄 1)과 (줄 2)에서 "[ ]"은 해당 propertyname(여기서는 diary와 ShowHiddenHandles을 의미한다.)에서 이용 할 수 있는 지정어(즉, propertyvalue)이다. 다시 말해서, 해당 propertyname을 제어 할 수 있는 지정어로서 다른 비슷한 용어를 사용하면 안되고, 반드시 지정해준 "on, off"만 사용하여야 한다. 그런데, (줄 2)에는 "{ }"으로 "off"라는 지정어(propertyvalue)를 둘러싸고 있는데, 이것은 "default propertyvalue"를 의미한다. 즉, Matlab이 자동으로 propertyvalue를 설정한 경우를 의미한다. 물론, "set.m"함수로 default propertyvalue"를 바꿀 수도 있다.

```
□get(0,'units') ↲          %propertyname "units"에 설정된 "propertyvalue"를
                           %"get.m"함수를 이용하여 얻는다.
ans =
pixels                     %propertyname인 "units"에 해당하는
                           %default propertyvalue이다.
                           %"units"에 대해서는 "figure object"에서 언급하겠다.
```

```
□set(0,'units','nor') ↲        %"propertyvalue"를 "nor"로 "set.m"함수를
                               %이용하여 바꾼다.
□get(0,'units') ↲              %"propertyvalue를 다시 확인한다.

ans =

normalized
```



주의 propertyname은 대·소문자를 구분하지 않으며, 또한, "normalized"를 "nor"와 같이 약식 표현해도 된다. 단, "nor"로 시작되는 다른 propertyname이 없다는 전제하에서 가능하다. 만일, 여러분이 propertyname에대해 많이 알고 있다면 상관없는데, 그렇지 안다면, 약식 표현은 자칫 error의 원인이 되므로 사용하지 말 것을 권한다.

(줄 1)에는 Matlab이 자동으로 설정해준다는 표현인 "{ }"이 없다. 또한, 여러 개의 항들에서 "{ }"이 없는 항들을 볼 수 있다. 이것은 Matlab 회사에서 잘못표시 한 것으로서 반드시 모든 propertyname에는 default propertyvalue가 존재해야 한다. 그 사실은 Matlab에서 제공하는 help manual을 보면 알 수 있을 것이다. 사실, (줄 1)은 "Diary: [on | {off}]"로 표시해야 옳은 표현이다.

마지막으로 박스 안에 표시된 것과 같이 AutomaticFileUpdates과 ButtonDownFcn사이에 한 줄을 띄운 것을 볼 수 있다. 한 줄띄운 아래에 나오는 모든 propertyname은 "**universal properties**"를 의미한다. 즉, 모든 object가 다 갖고 있는 공통된 properties이다. 그런데, ButtonDownFcn은 옆에 아무런 지정어나, 또는 사용할 수 있는 format(예를 들어서 "string", "number" 등등)이 없다. 그것은 현 object인 Root Object에서는 사용 할 수 없는 propertyname을 의미한다.

```
                ..... 생략 .....
AutomaticFileUpdates: [ on | off ]


ButtonDownFcn
                ..... 생략 .....
```

나. figure object

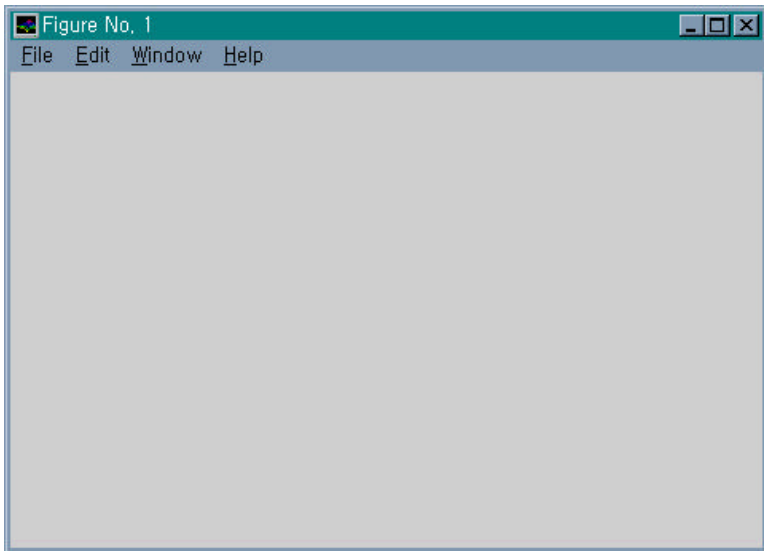
간단한 예제를 다루어 가며 설명하겠다.

□h=figure ↲

```
h =

     1
```

위와 같은 명령을 실행하면, [**그림**3-6]**과 같은** figure object**가 생성 될 것이다**.



[**그림** 3-6] original figure**의 모양**

이제, [**그림** 3-6]**을** [**그림**3-7]**과 같이 변형하여 보자**.



[그림 3-7] 변형된 figure의 모양

우선, [그림 3-6]과 비교하여, 이전에 언급한 "numbertitle"을 없애고(propertyvalue는 "off"), "name"을 "By Lim Jong_Su"로 바꾸었다. 또한, figure의 크기를 바꿀 수 없도록 "figure를 확대시켜 주는 icon"이 사라졌다.

figure의 크기를 마음대로 조정 할 수 있고, 없고는 "**resize**"라는 propertyname이 결정한다. 지정어로는 "[{on} | off]"가 있다.
그리 중요한 예제는 아니지만 "**set.m**"함수를 이용하는 방법에 익숙해진다는 생각으로 한 번 해보기 바란다.

"**set.m**"함수와 "**get.m**"함수는 주어진 객체의 handles를 조정하는데, 필수적인 함수이다.

```
□set(h,'numbertitle','off','name', ...      %이전에 설명한 " ... "명령어가 기억나는가 ?
'ByLime Jonfg_Su','menubar','none','resize','off') ↲
```

이번에는 [그림3-6]에 주어진 figure의 크기를 알아보고, Matlab의 "units" 체계에 대해서도 알 보자.

```
□fig_size=get(h) ↲

fig_size =

    165    114    470    279

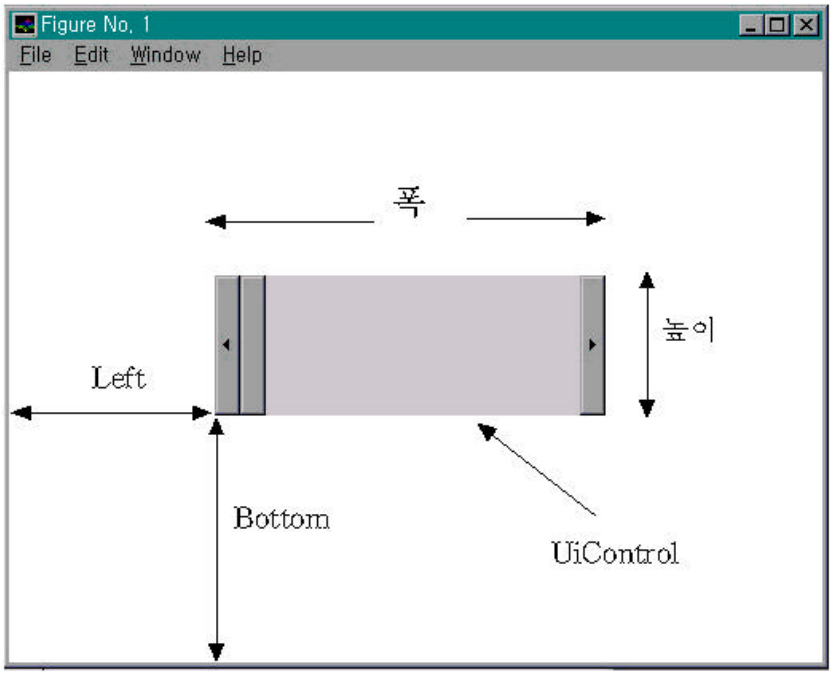
□get(h,'units') ↲

ans =
pixels
```

주어진 그림의 크기를 알기 위해서 figure object의 propertyname중 "position(약식 표현:'pos')"를 이용하였다.

Matlab에서 figure의 크기는 왼쪽 하단을 원점으로 하여 폭과 높이로 정해진다. 즉, **[left bottom width height]**와 같은 형식을 갖는다.

앞으로 여러분이 GUI프로그램을 작성하는 경우, "slider, radio button, push button 등등"은 그림에서 차지할 위치를 지정해 주어야 한다. 그때, 사용되는 위치 지정법이기도 하다.
자세한 사항은 [그림 3-8]을 참고하기 바란다.



[그림 3-8] figure의 위치 지정법

그러므로, [그림 3-6]의 크기인 fig_size의 해당 숫자는 각각 다음과 같다.

Left	165
Bottom	114
width	470
Height	279

[표 3-1] figure의 위치

Matlab에서 사용 할 수 있는 units의 종류로는 5가지가 있다.

a)**inches** b)**centimeters** c)**normalized** d)**points** e)**{pixels}**

또한, Matlab에서 지원되는 모든 units체계는 window의 왼쪽하단([그림3-8]참고)을 원점으로 정하고 측정하는데, Normalized units는 왼쪽 하단이 (0,0)에 해당하며, 오른쪽 상단은 (1.0 1.0)에 해당한다.
모든 객체는 각각 units를 정의한 뒤에 생성할 수 있다.

보통 출판업 종사하는 분들이 즐겨 쓰는 units인 "**points**"는 1/72inches에 해당한다.

다. Axes object

만일, 여러분이 여러 개의 객체를 생성했다고 하자! 이때, 원하는 객체의 특성을 바꾸고 싶다면, 직접 해당 객체의 handle value를 반환하는 functions를 이용하면 쉬울 것이다.

1. gcf (get current figure) : 현재 active 상태인 figure handle을 반환한다.


용어해설

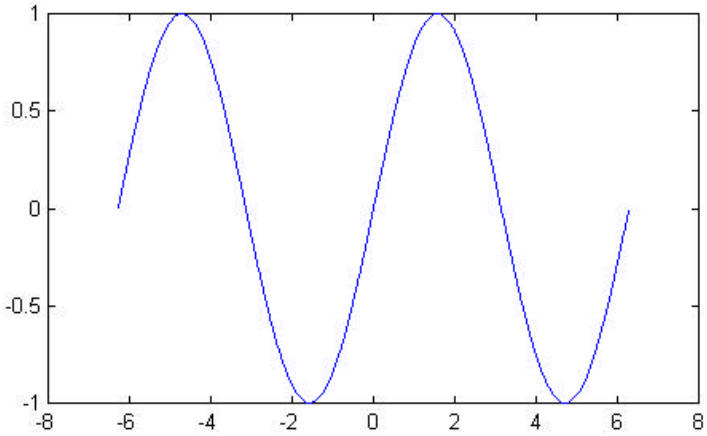
active 상태

만일, 여러분이 여러 개의 figure 객체를 만들었다고 해도, mouse로 한 번에 클릭 할 수 있는 figure는 하나뿐이다. mouse로 클릭된 figure의 numbertitle부는 녹색을 띄며, (그림[3-8]참고) 이와 같은 상태를 **active** 상태라고 한다.
객체의 특성을 바꾸려면 객체가 속해 있는 figure가 active 상태이어야 한다.

2. gca (get current axes) : active 상태의 figure안에 있는 axes의 해당 handles를 반환한다.

```
t=-2*pi:1/100:2*pi; ↲
y=sin(t); ↲
plot(t,y) ↲
```

위와 같은 일련의 명령어를 실행하면, [그림 3-9]와 같은 figure가 생성 될 것이다.

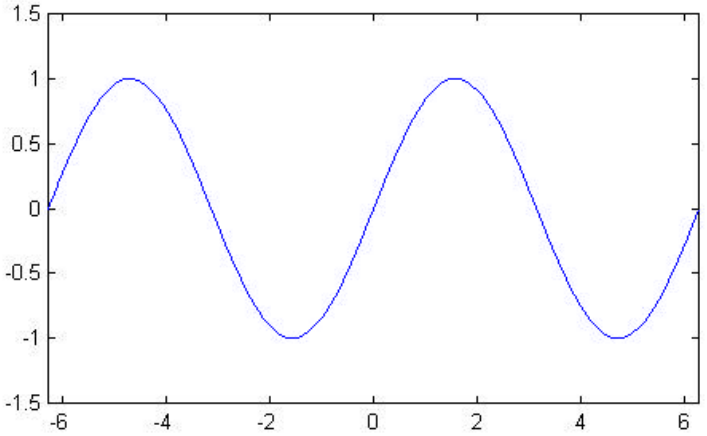


[그림 3-9] sample figure

[그림3-9]에서 알 수 있듯이 line의 끝이 좌표축에 붙지 않고 있다.

Axes Object	
propertyname	propertyvalue
xlim, ylim, zlim	**각각의 좌표축에 대한 경계를 정한다.**

□set(gca,'xlim', [min(t) max(t)],'ylim',[-1.5 1.5]) ↲
%항상, 최소 값이 먼저 나와야 한다.

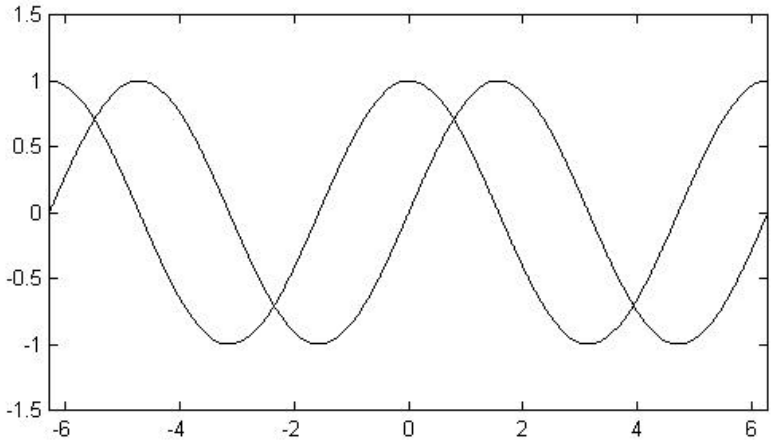


[그림 3-10] sample figure에 대한 좌표축의 경계 변환

3. gco (get current object) : active figure**에서 가장 최근에 생성된 객체 또는**, mouse**로 가장 최근에 클릭 된 객체의** handle**을 돌려준다**.

```
□t=-2*pi:1/100:2*pi; ↲
□y1=sin(t); ↲
□y2=cos(t); ↲
□plot(t,y1,'k',t,y2,'k') ↲      %'k'는 검은 색을 의미한다.
                                 %"plot.m"함수에 대해서는 앞에서 배웠다.

□set(gca,'xlim', [min(t) max(t)],'ylim',[-1.5 1.5]) ↲
```



[**그림** 3-11] sample figure

[**그림** 3-11]**에서 보여준 것처럼, 여러 개의 선들이 같은 색으로 지정되어 있다든지 또는, 어느 한 선을 강조해서 표현하고 싶을 때가 있을 것이다. 그렇다면, 먼저 강조하고 싶은 선을** mouse**로 클릭한다.**



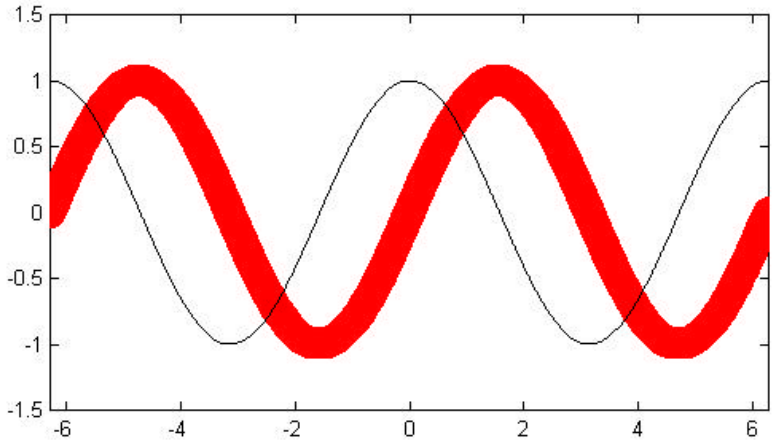
주의 mouse**로 객체를 클릭하여 선택하려면, 객체를 구성하는 각각의 모서리로부터** 5 pixles **바깥쪽까지 클릭한 것이 유효하다.** 5 pixels**를 벗어나서 클릭하면, 해당 객체 선택이 안 된다.**

이제, 강조하고 싶은 선을 클릭하였다면, 다음과 같은 명령어를 사용하라. 그러면, 클릭한 선의 색이 붉은 색으로 바뀐 것을 볼 수 있을 것이다. 또한, 선의 두께도 굵어 진 것을 알 수 있을 것이다.

Line Object	
propertyname	propertyvalue
Color	**선의 색을 변경하여 준다.**
Linewidth	number

□set(gco,'color','r','Linewidth',15) ↲

위와 같은 명령을 실행하면, [그림3-11]이 [그림3-12]와 같이 클릭한 선의 모양만 바뀐 것을 알 수 있다. 이 기법은 **buttondown** 종류의 propertyname과 연결해서 GUI프로그램작성 시 자주 이용된다.



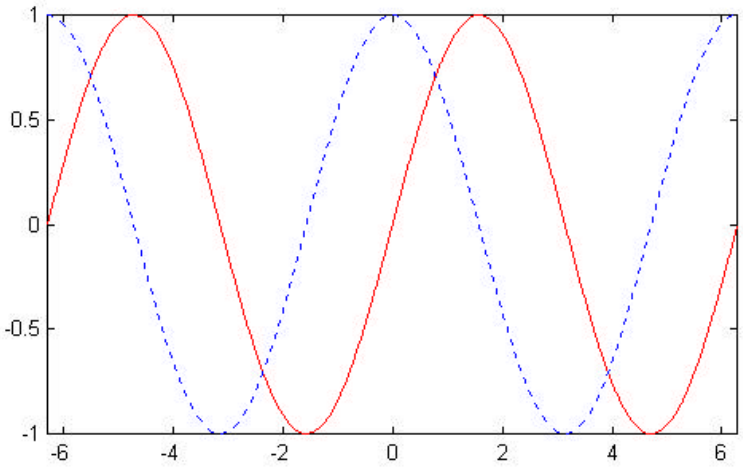
[그림 3-12] gco함수의 효과

여러분 중에서, Matlab의 "hold on/off"명령을 아시는 분이 계실 것이다. 물론, 현재의 그래프와 앞으로 만들 그래프를 겹쳐 표현하는 데 있어서, 사용하기에 아주 간단한 함수이다. 그러나, GUI 프로그램 작성 할 때는 적절하지 못한 함수이다.

Axes Object	
propertyname	propertyvalue
Nextplot	간단히 설명하면, "add"인 경우에는 "hold on"효과를, "replace"이면, "hold off"의 효과를 준다.

□t=-2*pi:1/100:2*pi; ↲
□y1=sin(t); ↲
□y2=cos(t); ↲
□plot(t,y1,'r') ↲
□set(gca,'nextplot','add','xlim',[min(t) max(t)]) ↲
□plot(t,y2,'b:') ↲

위와 같은 일련의 명령을 실행하면, [그림 3-13]을 얻을 수 있다.



[그림 3-13] Axes Object의 nextplot 이용법


주의 figure object에 있는 "nextplot"과는 다른 기능을 한다는데 주의하라!

다. text object :

원칙적으로는 "uicontrol, uimenu"objects를 다루어야 하지만, 이들 objects에 대해서는 부록에 실은 "GUI 프로그램 작성법"을 참고하기 바란다. 또한, 다른 "line, path, image, surface, light" objects에 대해서는 다루지 않겠다. image object를 제외한 objects들은 고급 그래픽 interface에서나 이용하는 방법이고, 별로, 중요하지 않다고 생각하기 때문이다.

[그림 3-1]의 (g)text object는 "한글"로 작성 한 것이 아니라, Matlab을 이용하여 작성 한 것이다. 이처럼 여러 가지 특수 symbol을 그래픽에 첨가해야 하는 경우가 자주 있다. 이때, 사용되는 대표적인 함수가 "text.m"함수 이다.

전형적인 사용법(syntax) : **text.m**

(a)"**text_handle=text(x,y,'string')**"

→주어진 좌표축에 기반을 두고, 좌표(x,y)에 "string"을 display한다.

(b)"**text_handle=text(x,y,z,'string')**"

→주어진 좌표축에 기반을 두고, 3차원 좌표(x,y,z)에

→"string"을 display한다.

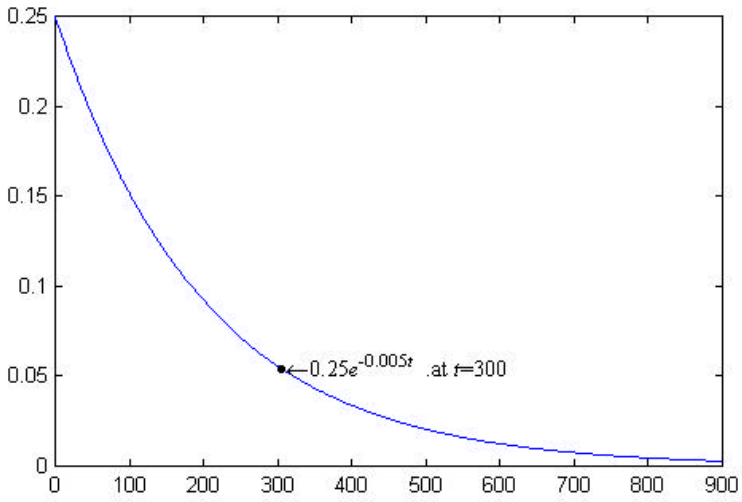
(c)"**text_handle=text(x,y,z,'propertyname','propertyvalue')**"

→주어진 좌표(x,y,z)에 특성을 첨가하거나, 바꾸어 준다.

→2차원인 경우에는 (x,y)만 설정해 주면 된다.

"**text.m**"함수의 여러 가지 특성 중에서 여기서는 특수 symbol을 이용하는 방법만 알아보겠다.

[그림3-14]를 통하여 설명을 전개해 나가겠다.



[그림 3-14] 특수 symbol 사용법

[그림 3-14]를 만들기 위해서는 다음과 같은 일련의 명령어가 필요하다.

```
t=0:900; ↲
plot(t,.25*exp(-.005*t)) ↲
h=text(300,.25*exp(-.005*300),...
'\bullet\leftarrow\fontname{times}0.25{\ite}^{-0.005{\itt}}  .at  {\itt}=300');
```

"**\bullet, \leftarrow**"가 무엇을 의미하는 지는 [표3-2]를 참고하면 알 수 있을 것이다.

Character Sequence	Symbol	Character Sequence	Symbol	Character Sequence	Symbol
₩alpha	α	₩upsilon	υ	₩sim	~
₩beta	β	₩phi	φ	₩leq	≤
₩gamma	γ	₩chi	χ	₩infty	∞
₩delta	δ	₩psi	ψ	₩clubsuit	♣
₩epsilon	ε	₩omega	ω	₩diamondsuit	♦
₩zeta	ζ	₩Gamma	Γ	₩heartsuit	♥
₩eta	η	₩Delta	Δ	₩spadesuit	♠
₩theta	θ	₩Theta	Θ	₩leftrightarrow	↔
₩vartheta	υ	₩Lambda	Λ	₩leftarrow	←
₩iota	ι	₩Xi	Ξ	₩uparrow	↑
₩kappa	κ	₩Pi	Π	₩rightarrow	→
₩lambda	λ	₩Sigma	Σ	₩downarrow	↓
₩mu	μ	₩Upsilon	Υ	₩circ	∘
₩nu	ν	₩Phi	Φ	₩pm	±
₩xi	ξ	₩Psi	Ψ	₩geq	≥
₩pi	π	₩Omega	Ω	₩propto	∝
₩rho	ρ	₩forall	∀	₩partial	∂
₩sigma	σ	₩exist	∃	₩bullet	•
₩varsigma	ζ	₩ni	₩θ	₩div	÷
₩tau	τ	₩cong	≅	₩neq	≠
₩equiv	≡	₩approx	≈	₩aleph	ℵ
₩Im	ℑ	₩Re	ℜ	₩wp	℘
₩otimes	⊗	₩oplus	⊕	₩oslash	∅
₩cap	∩	₩cup	∪	₩supseteq	⊇
₩supset	⊃	₩subseteq	⊆	₩subset	⊂
₩int	∫	₩in	∈	₩o	o

[표3-2] 특수 문자표

"한글"에서 수식을 넣는 방법과 거의 같다.

이들 문자들은 "Latex"문자 체계들로써, text object의 interpreter가 "tex"인 경우에만 특수 문자로 변환된다.

□get(h,'interpreter') ↲

ans =

tex

만일, 다음과 같이 명령을 하면, 'string'이 특수문자로 변환되지 않고, typing한 그대로 figure안에 display된다.

□set(h,'interpreter','none') ↲

또한, figure에 display되는 문자의 "font"도 바꾸어 줄 수 있다.

□h=text(300,.25*exp(-.005*300),...
'\bullet\leftarrow\fontname{times}0.25{\ite}^{-0.005{\itt}} .at {\itt}=300');

font의 종류	주어진 font에 대한 설명
\bf	Bold Font
\it	italics font
\sl	oblique font(거의 이용 안 함)
\rm	normal font
\fontname{fontname}	위에 주어진 font외의 다른 font들

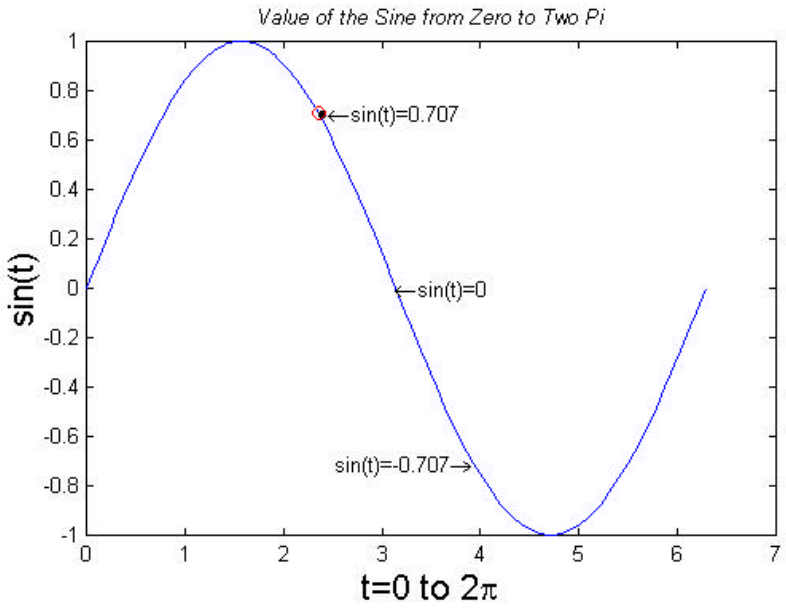
[표 3-3] font의 종류

이들 특수 문자 지정어들은 "{ }"으로 구분한다.
또한, 아래첨자는 " _ "으로 만들고, 윗 첨자는 " ^ "으로 만든다.


주의

"\, {, }, _, ^"문자들은 모두 앞에 " \ "을 첨가하면 display 될 수 있다.
즉,
"\\, \{, \}, _, \^"의 형태를 취하면 된다.

이번에는 문자의 정렬방법에 대해서 알아보자.
[그림 3-15]는 sample용 figure이다.



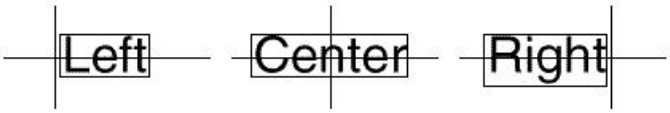
[그림 3-15] 문자열의 정렬 방법

문자열(string)의 정렬을 관장하는 propertyname으로는 "**HorizontalAlignment**, **VerticalAlignment**" 2가지가 있다.
[표 3-4]와 [그림3-16]을 참고하면 쉽게 이해 할 수 있을 것이다.

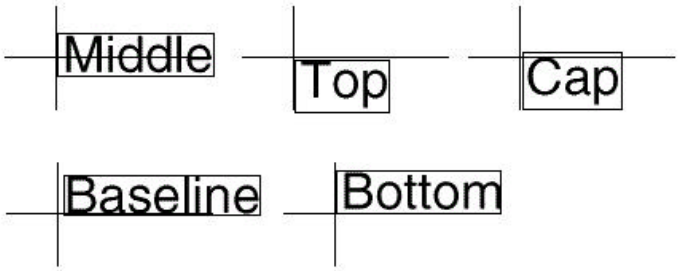
Text Object	
propertyname	propertyvalue
HorizontalAlignment	{left} | center | right
VerticalAlignment	top | cap | {middle} | baseline | Bottom

[표 3-4] 문자열의 정렬방법

[그림3-16 (a)]는 **HorizontalAlignment**에 대한 설명이고, [그림3-16(b)]는 **VerticalAlignment**에 대한 설명이다.



(a) "HorizontalAlignment"의 경우.



(b) "VerticalAlignment"의 경우.

[그림3-16]문자열의 정렬 방법

그런데, "Text.m"함수에서 지정한 정확한 좌표에 문자열이 위치하는 것은 아니라 약간, 벗어난 지점에 문자열이 나타난다. 왜? 약간 벗어난 위치에 문자열이 나타나는 지, 그 이유에 대해서는 mathworks.com에서 언급하지 않은 것으로 알고 있다.
그러나, 정확한 좌표에서의 이탈양은 상당히 작으므로 여러분은 신경 쓰지 않아도 될 것이다.

Matlab 명령어	해당 명령어의 설명
zoom on	확대시키고 싶은 영역에서 mouse의 왼쪽 버튼을 click하면 화면이 확대된다. 오른쪽 버튼을 누르면 축소된다.
zoom off	Zoom기능을 없앤다.
zoom xon	x축에 대해서만 확대를 한다. mouse 버튼 기능은 "zoom on"과 같다.
zoom yon	x축에 대해서만 확대를 한다. mouse 버튼 기능은 "zoom on"과 같다.
zoom out	원래의 figure로 돌아간다.

[표 3-5] 화면의 확대 축소 기능

[표3-5]에서 언급한 "**zoom.m**"함수를 이용하여, [그림 3-15]에서 나타낸 작은 원과 점(dot) 부분을 확대하여 본다. 그러면, 점이 작은 원의 중심이 아니라는 것을 알 수 있을 것이다.

[표3-4]와 [그림 3-16]을 참고하면, 여러분이 직접 [그림3-15]를 생성하는 M-file을 작성 할 수 있다고 본다.

[그림 3-17]은 [그림 3-15]를 만드는 데, 이용한 M-file이다.
파일이름은 "**fig3_17.m**"이다.

```
%
%[그림 3-15]를 만드는 데, 이용되는 M-file.
%'text.m"함수의 이용 방법.
%
%1998.2.18   By Lim Jong_Su
%

close all
clear all

t=0:pi/100:2*pi;
y=sin(t);
plot(t,y)
xlabel('t=0 to 2\pi','fontsize',16)
ylabel('sin(t)','fontsize',16)
title('\it{Value of the Sine from Zero to Two Pi}')
set(gcf,'color','w')
text(3*pi/4,sin(3*pi/4),'\bullet\leftarrowsin(t)=0.707')
set(gca,'nextplot','add')
plot(3*pi/4,sin(3*pi/4),'ro')
text(pi,sin(pi),'\leftarrowsin(t)=0')
text(5*pi/4,sin(5*pi/4),...
   'sin(t)=-0.707\rightarrow','Horizontalalignment','right')
set(gca,'nextplot','replace')
```

[그림 3-17] "fig3_17.m"파일 내용



주의 만일, 여러분이 "figure Object, Axes Object"처럼 상위 Level의 default property value를 바꾸었다면, "fig3_17.m" 파일의 맨 아래 줄에 있는 "**set(gca,'nextplot','replace')**"와 같이, 다시 default propertyvalue로 환원 해주는 것을 잊지 말자!
만일, 환원해 주지 않으면, 앞으로 만들어지는 모든 Axes Object의 하위 Level과 같은 Axes level의 함수들은 동일한 화면에 그림들을 그려주기 때문이다.

지금까지 여러분은 Matlab Graphics system중 Handle graphics에 해당하는 Low_Level functions에 대해서 알아보았다.

이제, 좀 쉬운 High_level functions에 대해서 알아보자!

라. High_level functions.

High_level functions에 속하는 그래픽 함수들은 일반적으로 데이터를 화면에 display하기 위해 사용되는 함수들이다.

상당히 많은 종류의 그래픽 함수들이 존재하는 데, 이들 함수들은 Low_level functions처럼 데이터를 display하는데, 필요한 여러 가지 propertyname /propertyvalue를 묻지 않고, 자동으로 좌표의 크기 조절(axis scaling)과 선의 색(Line color)등을 설정해준다.

본 단원에서는 그래픽 함수들의 특성에 따라서 3가지 부류로 나누어 이들 그래픽 함수들을 다루고자 한다.

1. 다음과 같은 대표적인 그래픽 함수들을 설명하겠다.
   "plot.m, plot3.m, contour.m, contour3.m, contourf.m, clabel.m, stem.m, stem3.m, stairs.m, compass.m, feather.m, comet.m, comet3.m,scatter.m, scatter3.m, polar.m, rose.m, "

2. 다음과 같은 대표적인 3 차원 그래픽 함수들을 설명하겠다.
   "surf.m, surfc.m, mesh.m, meshc.m, meshz.m, view.m, meshgrid.m, colormap.m, colorbar.m"

3. 그 밖의 유용한 그래픽 함수들을 설명하겠다.
   "pie.m, pie3.m, bar.m, barh.m, bar3.m, bar3h.m, area.m, errobar.m, ezplot.m"

그밖에도 하나의 그래프를 완성하는데 필요한, "legend.m, subplot.m, 등등"의 설명도 첨부해 나갈 것이다. 그러나, 모든 함수들을 자세히 설명하지는 않을 것이다. 몇몇 대표적인 함수들만 자세히 강의하고, 나머지는 그 사용법만 간단히 답습해나가는 식으로 하겠다.

위에 열거한 그래프 함수들 정도면, 충분히 모든 상황을 표현할 수 있다고 본다.

1. 일반적으로 많이 이용되는 그래픽 함수들 :

가) plot.m : 제 1 부류

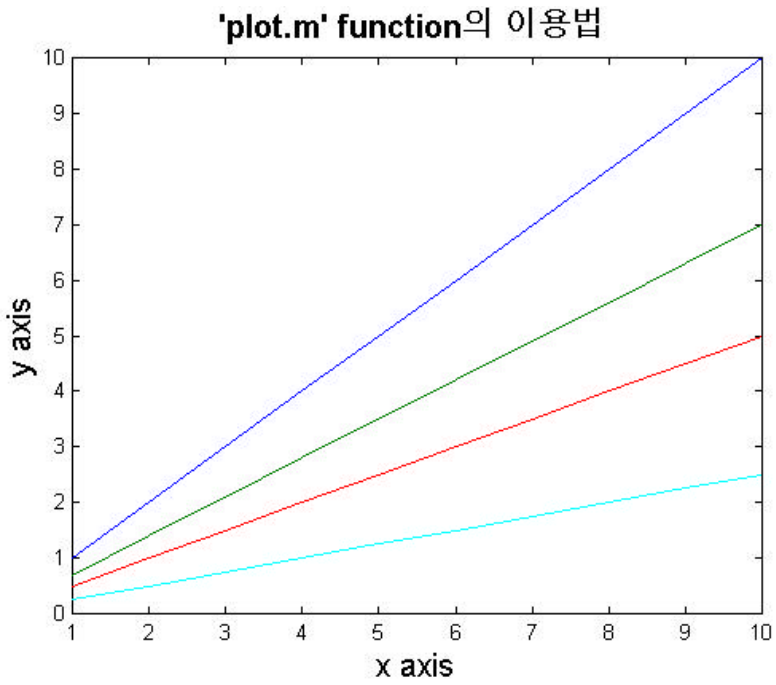
지금까지 "plot.m"함수는 많이 이용하여 왔다. 그런데, 이번에는 좀 더 자세히 그 이용법을 알아보겠다.

전형적인 사용법(Syntax) : **plot.m**

a) **plot(Y)**
→**만일**, Y**가 실수이면**, x**축은** Y**의** index**이고**, y**축은** Y**의 열 벡터들로**
→**이루어진다**. **그러나**, Y**가 복소수이면**, x**축은** real(Y), y**축은**
→imag(Y)**로 이루어진다**.
b) **plot(X1,Y1,...)**
→X1**의 열 벡터와** Y1**의 열 벡터가 각각의** line**을 구성한다**.
c) **plot(X1,Y1,LineSpec,...)**
→LineSpec**은** color, linestyle, maker**로 구성되어 선의** style**을**
→ **결정한다**.
d) **plot(...,'PropertyName',PropertyValue,...)**
→"Line"object**에 있는** propertyname/propertyvalue**로 선의** style**을**
→**결정한다**.
e) **h = plot(...)**
→**여러 개의** Line**들이 있는 경우**, **각각의** Line**에 할당된**
→handle value**가 열 벡터의 모양으로 반환된다**.

□X=[1:10; 0.7*[1:10]; 0.5*[1:10]; 0.25*[1:10])'; ↲
□plot(X) ↲
□xlabel('x axis','fontsize',15) ↲
□ylabel('y axis','fontsize',15) ↲
□title('\bf{''plot.m''**함수의 이용법**}','fontsize',15) ↲

위와 같은 일련의 명령어를 실행하면 어떻게 그래프가 만들어지겠는가?



[그림 3-18] "**plot.m**"함수의 이용법

[그림3-18]에서 알 수 있듯이 행렬 X에 대한 각각의 원소들이 Y축을 이루고 있고, 행렬 X에 대한 index가 X축을 이루고 있다.(즉, x=1:length(X))

만일, 여러분이 선(line)들의 색이나 type을 정해주지 않으면, 자동으로 default값에 의해 결정되어 진다.
이들 선들의 색에 대한 default값은 다음과 같은 방법으로 얻을 수 있다.

```
□Line_Color =get(gca,'colororder') ↲

Line_Color =

         0         0    1.0000
         0    0.5000         0
    1.0000         0         0
         0    0.7500    0.7500
    0.7500         0    0.7500
    0.7500    0.7500         0
    0.2500    0.2500    0.2500
```

또한, 선들의 type에 대한 default값은 다음과 같은 방법으로 역시 얻을 수 있다.

□Line_Type =get(gca,'Linestyleorder') ↲

Line_Type =

-

이들 Line_Color 행렬과 Line_Type 벡터의 조합으로 모든 선(Line)들의 style이 결정된다.
결정 방법은 다음과 같다.
(a) Line_Type의 첫 번째 원소와 선들의 색에 대한 행렬인 Line_Color의 모든 색 을 순환하며 각각의 선들을 구성한다.
(b) Line_Type의 두 번째 원소와 선들의 색에 대한 행렬인 Line_Color의 모든 색 을 순환하며 각각의 선들을 구성한다.
..... 반 복 .....

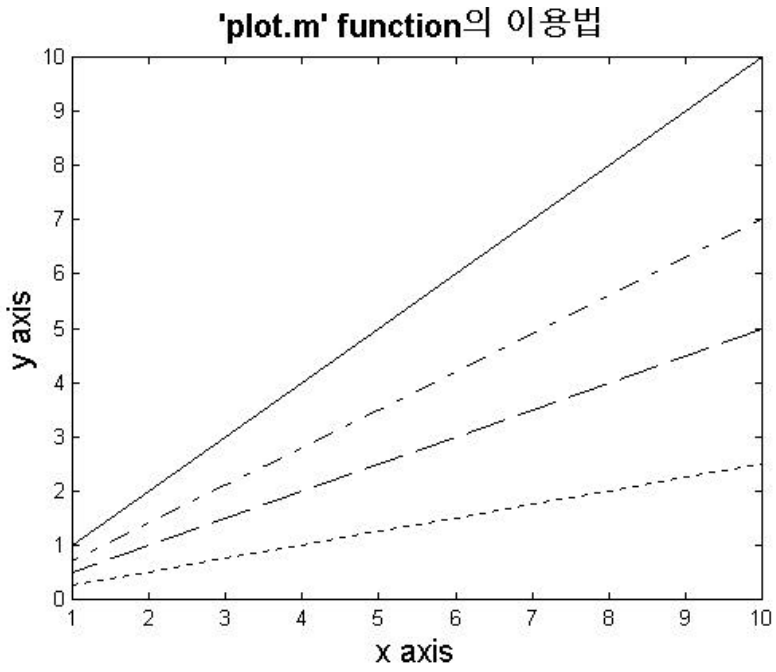
그런데, 대부분의 학생들은 컬러 프린터기를 사용하지 못하고 있다고 본다. 그러므로, "plot.m"함수에 의해서 그려지는 선들의 색은 검은 색으로 일정하고 type만 바꾸어 가며 line을 그리기를 원할 것이다. 그러기 위해서는 default값인 "Line_Color, Line_Type 배열의 원소들"을 바꾸어 주어야한다.

□set(0,'DefaultAxesColororder',[0 0 0],'DefaultAxesLineStyleOrder','-|-.|--|:') ↲

위와 같은 명령어를 실행하고, 다시 [그림3-18]을 만드는 일련의 명령어를 똑같이 실행하면 [그림 3-19]와 같은 figure가 만들어 질 것이다.


주의

앞에서 보여준 "set.m"함수는 Root Object에 대한 propertyname/propertyvalue를 조정해주고 있다. Root Object는 최상위의 Level을 갖는 Object이다. 그러므로, matlab을 재 시동할 때까지는 모든 선들의 style이 위에서 결정된 사항을 따르게 된다. 또한, "**startup.m**"파일 안에 위와 같은 명령어를 삽입하면 matlab을 재실행하여도 선들의 style은 위에서 결정한 모양을 계속 따른다.
단! 윤곽선을 표현하는 "contour.m"함수 부류는 위에서 결정한 선들의 style을 따르지 않는다. 그것은 "contour.m"함수 내부에서 다시 선들의 style을 정의하기 때문이다.



[그림 3-19] 선들의 style에 대한 default값을 변경한 경우

여러분 중에서 데이터를 그래프로 표현하고자 하는 경우, 한쪽 데이터의 범위(range)는 일정한 경계 값으로 고정시키고, 다른 쪽은 데이터의 모든 양을 표현하고 싶을 때가 있을 것이다. 즉, 어느 구간만 보고 싶은 경우가 있을 것이다.

이처럼 어떠한 좌표의 한쪽은 경계치로 고정시키고, 다른 쪽은 모두 표현하고 싶을 때는 "**axis.m**"함수를 이용하여 보라. 물론, 어떠한 좌표의 양쪽을 일정한 경계치로 고정시킬 수도 있다.

앞에서, Axes object를 배울 때, propertyname 중에서 "**xlim, ylim, zlim**"에 대해서 학습하였다. 이것과 같은 기능을 하는 함수가 바로 "**axis.m**"함수이다.

전형적인 사용 방법(syntax) : **axis.m**

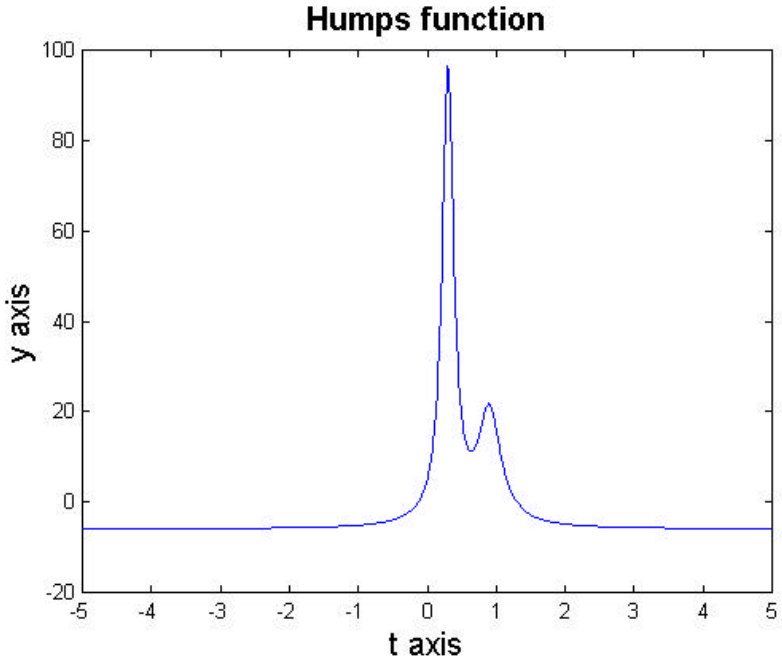
axis([xmin xmax ymin ymax zmin zmax])

→좌표의 범위(range)를 결정해준다.

→2차원 그래프인 경우에는 "zmin zmax"만 빼주면 된다.

```
t=-5:1/100:5; ↲
y=1/((t-0.3).^2+0.01)+1/((t-0.9).^2+0.04)-6; ↲   %y는 humps function.
plot(t,y) ↲
```

앞에서와 같은 일련의 명령어를 실행하면, [그림 3-20]을 얻을 수 있다.



[그림 3-20] "**axis.m**"함수의 사용 예

이제, t**축의 범위를** [0 ∞]**으로 바꾸고자 한다. 그렇게 하기 위해서는 다음과 같은 명령을 실행하면 된다.**

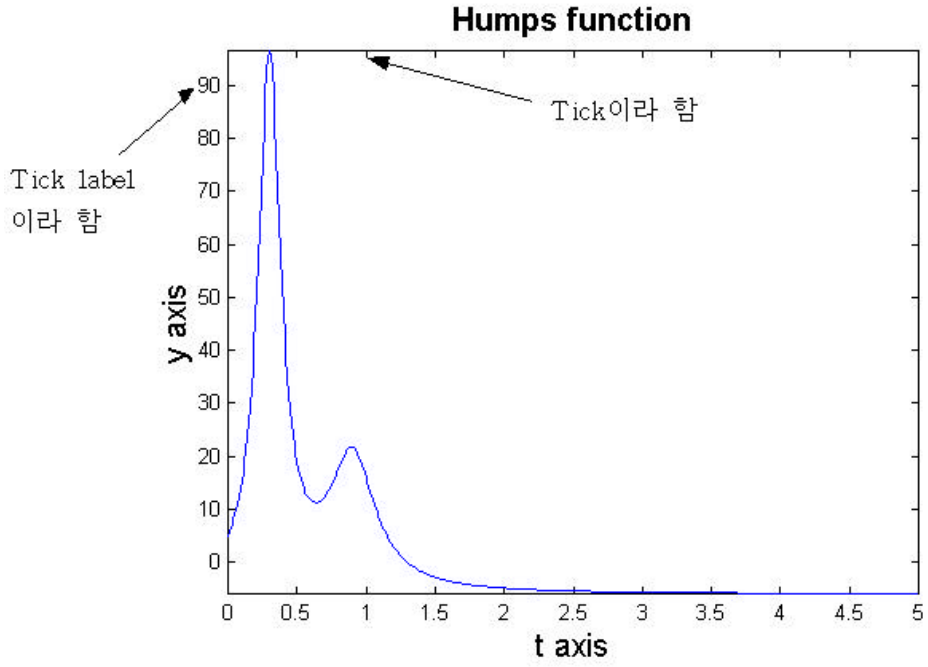
□axis([0 inf -inf inf]) ↲

[**그림** 3-21]**은** t**축의 범위를** [0 ∞]**으로 바꾼 경우의 그래프이다.**

앞에서 배운 Axes Object**의** propertyname/propertyvalue**를 이용하여** t**축의 범위를** [0 ∞]**까지 바꾸려면 어떻게 해야 하는가?**
필자의 생각으로는 여러분이 충분히 할 수 있다고 본다.

□set(gca,'xlim',[0 inf],'ylim',[-inf inf]) ↲

위와 같은 명령어를 실행하여도 [**그림** 3-21]**이 만들어진다.**



[그림 3-21] t축의 범위를 바꾼 경우

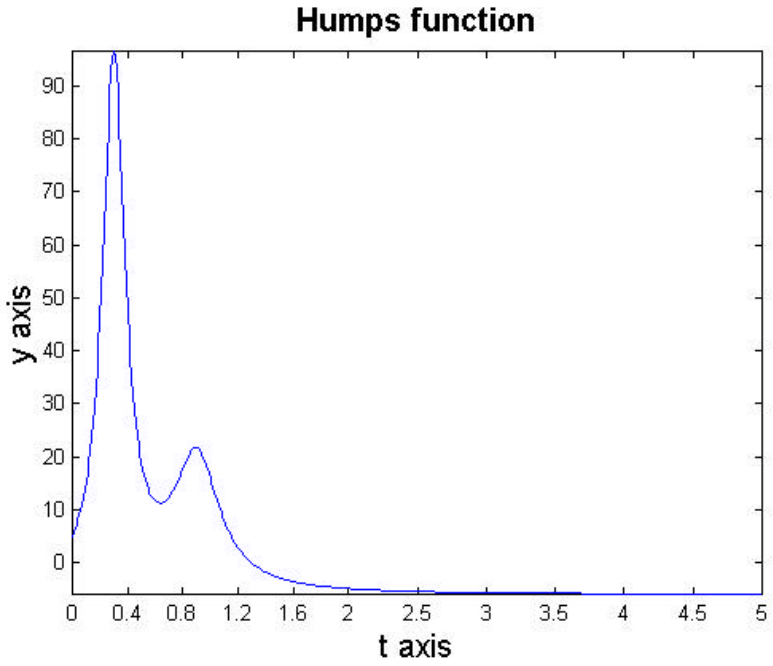
[그림 3-21]을 보면 데이터가 0≤ t ≤2 사이에 많이 분포하고 있는 것을 알 수 있다. 그러므로, 0≤ t ≤2 구간에서는 0.4간격마다 tick을 표시하고 싶다. 그러기 위해서는 다음과 같이 한다.

Axes Object	
propertyname	propertyvalue
xtick, ytick, ztick	Tick mark를 **위치시킬 수치** vector

□set(gca,'xtick',[0:0.4:2 2.5 3 3.5 4 4.5 5]) ↲

결과는 [그림 3-22]에서 볼 수 있다.

결국, "tick"의 위치는 불균등하게 놓을 수 있다.



[그림 3-22] Tick의 위치 변경 방법

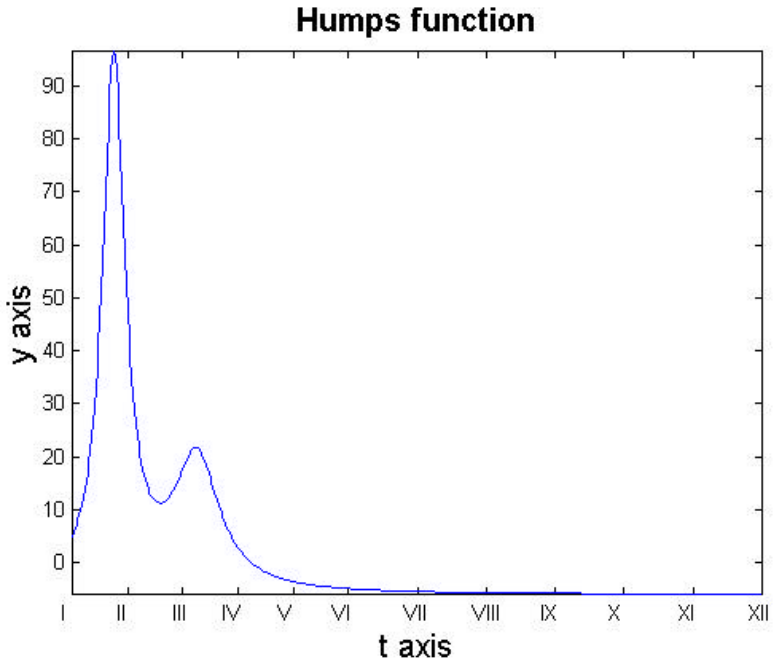
그런데, 만일, t축의 ticklabel(즉, 0, 0.4, 0.8 따위)을 수치가 아닌 문자열로 표현하고 싶을 때가 있을 것이다.

Axes Object	
propertyname	propertyvalue
xticklabel, yticklabel, zticklabel	Tick mark의 위치에 놓을 문자열(string)

x(y,z)ticklabel을 이용하는 경우 Tick mark들의 숫자만큼 문자열들이 있어야 한다. 그렇지 않으면 tick의 숫자만큼 문자열을 만들기 위해 다시 같은 문자열들을 사용한다.
[그림 3-22]에서는 t축에 있는 tick의 개수가 12개이다.

```
t_label=str2mat('I','II','III','IV','V','VI','VII','VIII','IX','X','XI','XII') ↲
set(gca,'xticklabel',t_label) ↲
```

위와 같이 실행하면 [그림3-23]과 같이 **xticklabel**이 로마숫자표시로 바뀔 것이다.



[그림 3-23] xticklabel의 변경 방법

알림	다음 시간에도 계속해서 High_level function에 대해서 알아보자.